

Aplicación Criptográfica: HTTPS

9 DICIEMBRE

Universidad Central de Ecuador

Creado por:

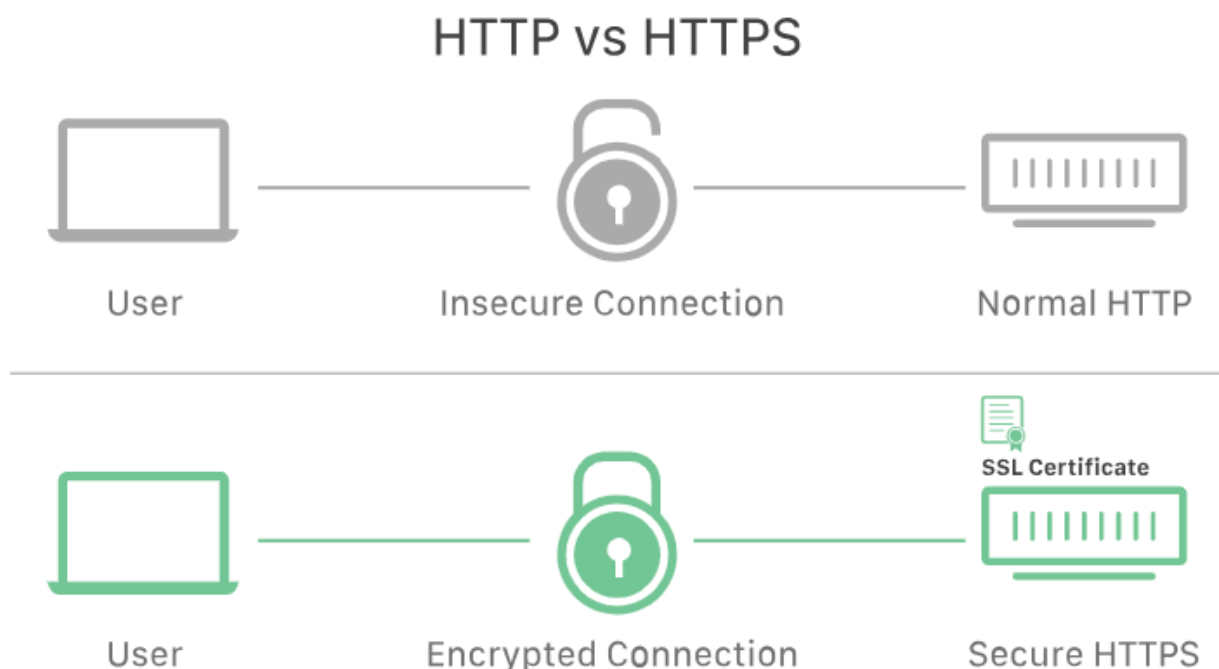
- Pineda Marco
- Ramírez Leonardo
- Vinuesa Edlith



Aplicación Criptográfica: HTTPS

Definición de la tecnología

HTTPS (Hypertext Transfer Protocol Secure) es la versión segura de HTTP, el protocolo principal utilizado para enviar datos entre un navegador web y un sitio web. HTTPS está encriptado para aumentar la seguridad de las transferencias de datos, lo cual es especialmente importante cuando los usuarios transmiten información confidencial, como al iniciar sesión en una cuenta bancaria o un servicio de correo electrónico [1].



Protocolos Criptográficos involucrados

Los protocolos criptográficos involucrados con HTTPS son principalmente SSL (Secure Sockets Layer) y TLS (Transport Layer Security). Estos protocolos aseguran las comunicaciones mediante el uso de infraestructura de clave pública asimétrica, lo que permite una transmisión segura de datos entre un navegador web y un servidor.

SSL (Secure Sockets Layer)

SSL es un protocolo de seguridad que proporciona privacidad, autenticación e integridad a las comunicaciones en Internet. Fue desarrollado por Netscape en 1995 y ha evolucionado a TLS (Transport Layer Security). SSL cifra los datos

transmitidos para que cualquier intento de interceptación resulte en una mezcla de caracteres ilegibles. Además, SSL inicia un proceso de autenticación llamado "handshake" para asegurar que ambos dispositivos sean quienes dicen ser.[2]

TLS (Transport Layer Security)

TLS es el sucesor de SSL y proporciona seguridad a las comunicaciones en Internet mediante la encriptación de datos entre aplicaciones web y servidores. TLS asegura la privacidad y la integridad de los datos, y es una práctica estándar en el diseño de aplicaciones web seguras. TLS utiliza un proceso de "handshake" similar al de SSL para establecer una conexión segura. [3]

Cómo funciona SSL/TLS en HTTPS:

Cuando un usuario se conecta a un sitio web mediante HTTPS, se realizan los siguientes pasos:

1. Apretón de manos: El cliente y el servidor realizan un apretón de manos para establecer una conexión segura. Durante este proceso, acuerdan los métodos de cifrado a utilizar e intercambian las claves criptográficas.
2. Cifrado: Una vez que se completa el protocolo de enlace, los datos transmitidos entre el cliente y el servidor se cifran mediante un cifrado simétrico, lo que garantiza que, incluso si los datos son interceptados, no se pueden leer.[4]
3. Comprobaciones de integridad: Los códigos de autenticación se utilizan para verificar que los datos no se hayan modificado durante la transmisión, lo que garantiza la integridad del mensaje. [5]

Criptografía de Hardware y Software que utiliza https

Criptografía en Hardware

1. Módulos de Seguridad de Hardware (HSM):
 - Utilizados para almacenar y manejar claves privadas de forma segura. Estos dispositivos están diseñados para evitar accesos no autorizados incluso en caso de intrusiones físicas.
 - Los HSMs son esenciales para la generación y gestión de claves de alta seguridad en servidores HTTPS.
 - Ejemplo: aceleradores de hardware que implementan algoritmos como RSA o ECDSA. [6][7]

2. Aceleración de Cifrado Simétrico:

- Procesadores modernos incluyen instrucciones como AES-NI para acelerar la encriptación y desencriptación con el Estándar de Cifrado Avanzado (AES).
- Estas optimizaciones mejoran significativamente el rendimiento al manejar grandes volúmenes de tráfico cifrado. [6][7]

3. Generadores de Números Aleatorios (RNG):

- Implementados en hardware para garantizar la generación de entropía segura. Son críticos para la creación de claves y valores aleatorios utilizados en protocolos TLS.
- Los RNG por hardware tienen una ventaja sobre los basados en software al ser menos susceptibles a predicciones o ataques. [6]

Criptografía en Software

1. Intercambio de Claves y Cifrado Asimétrico:

- HTTPS utiliza algoritmos como RSA y ECDSA para el intercambio de claves y autenticación del servidor.
- Estas operaciones están comúnmente implementadas en bibliotecas como OpenSSL o mediante la API Crypto del kernel de Linux. [7]

2. Funciones Hash Criptográficas:

- Algoritmos como SHA-2 se utilizan para garantizar la integridad de los datos transmitidos.
- Estas funciones suelen estar implementadas como bibliotecas en software, pero pueden ser aceleradas por hardware cuando está disponible. [6][7]

3. API Criptográficas:

- El kernel de Linux utiliza la API Crypto para facilitar operaciones criptográficas tanto en hardware como en software.
- Esta API permite a los desarrolladores usar algoritmos criptográficos sin preocuparse por si la implementación es en hardware o software. [7]

4. Bibliotecas de Software:

- Herramientas como OpenSSL, GnuTLS y LibreSSL proporcionan una implementación de protocolos TLS que incluye algoritmos de cifrado, intercambio de claves y gestión de certificados. [7]

Clasificación o tipos existentes

Los tipos existentes de HTTPS abarcan varios protocolos y mejoras dirigidas a asegurar las comunicaciones web. Estos protocolos no solo garantizan la integridad y confidencialidad de los datos, sino que también abordan los desafíos de seguridad emergentes. En las siguientes secciones se describen los tipos clave de HTTPS y sus funcionalidades.

Extensiones de seguridad

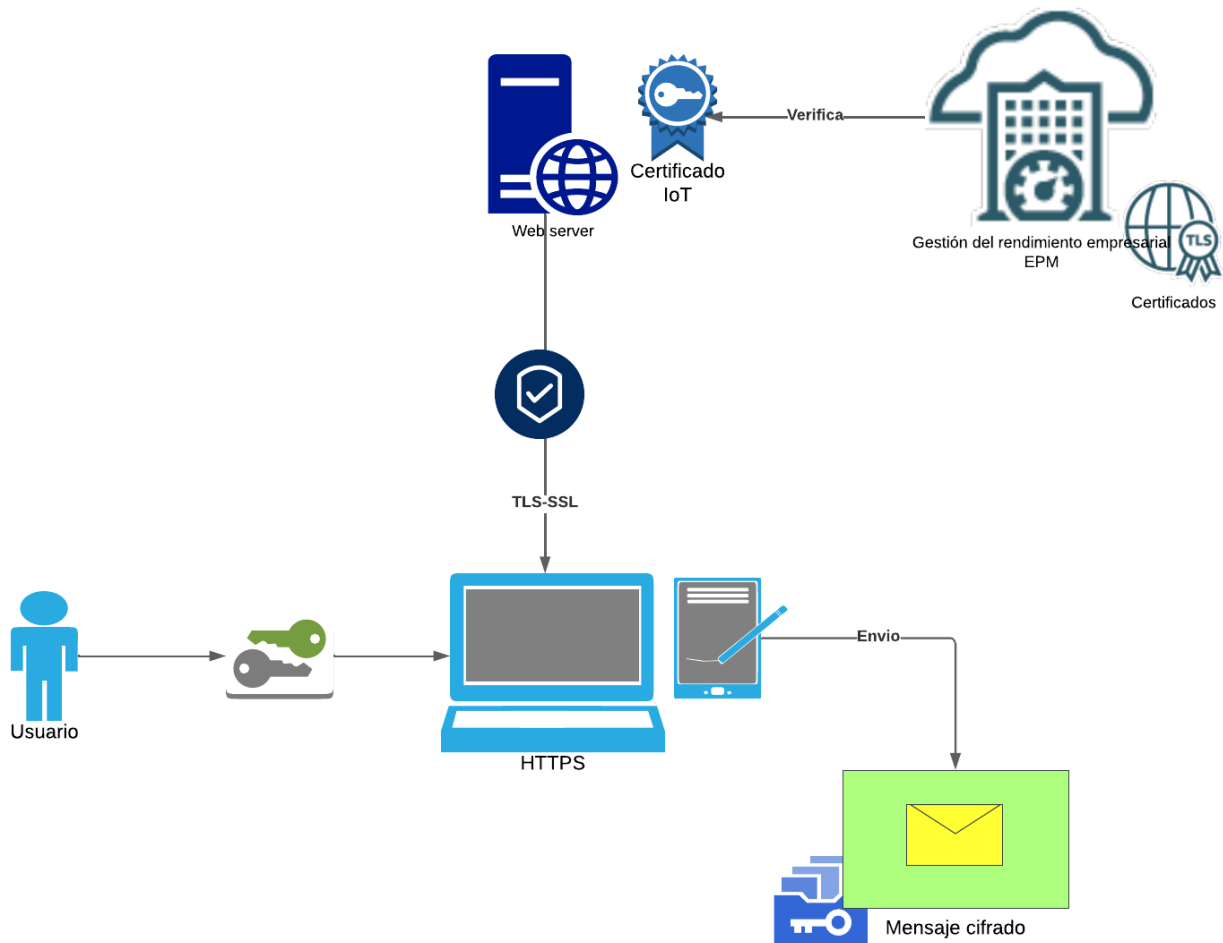
- **HTTP Strict Transport Security (HSTS):** obliga a los navegadores a conectarse únicamente a servidores usando HTTPS, mitigando los ataques de man-in-the-middle. [8]
- **Transparencia de Certificados (CT):** Tiene como objetivo evitar el mal uso de los certificados SSL proporcionando un registro público de todos los certificados emitido. [8]
- **HTTP Public Key Pinning (HPKP):** Permite a los sitios web especificar qué claves públicas son válidas para su dominio, reduciendo el riesgo de suplantación. [8]
- **HTTPS con ALPN (Application-Layer Protocol Negotiation):** Permite negociar la mejor versión de protocolo en aplicaciones HTTP/2 y HTTP/3.

Usos Frecuentes de HTTPS

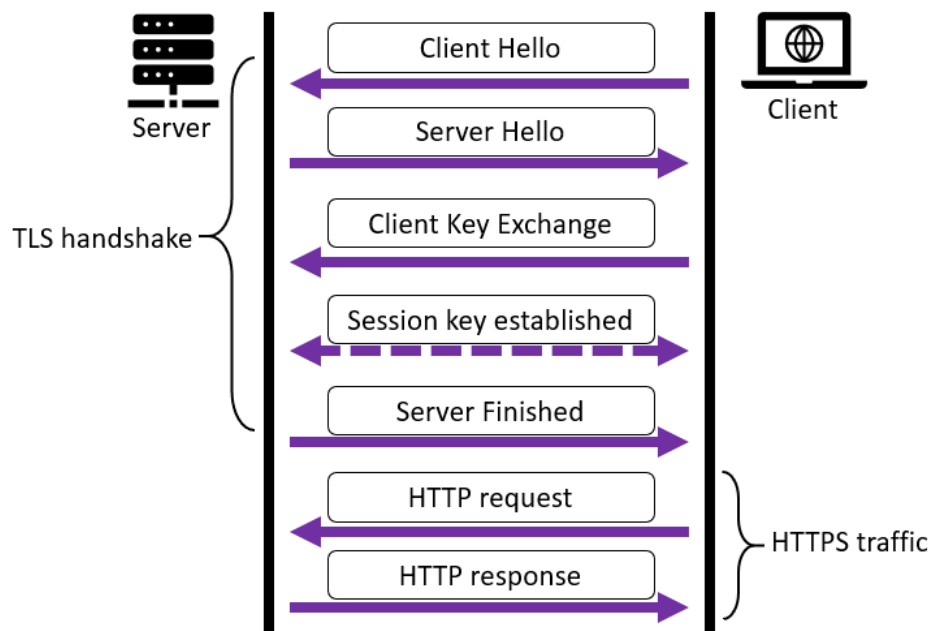
- **Comercio electrónico:** Tiendas en línea para proteger datos sensibles como información de tarjetas de crédito.
- **Plataformas bancarias:** Aseguran transacciones y datos financieros de los usuarios.
- **Sitios gubernamentales y empresariales:** Garantizan autenticidad y confidencialidad de la información.
- **Redes sociales y plataformas de comunicación:** Protección de datos de usuario y mensajes.
- **Aplicaciones SaaS (Software as a Service):** Protección de accesos y datos almacenados.

Diseño esquemático del funcionamiento

1. **Cliente (Navegador):** Inicia la conexión HTTPS.
2. **Servidor Web:** Proporciona el contenido cifrado.
3. **Certificado Digital:** Asegura la identidad del servidor.
4. **CA (Autoridad de Certificación):** Verifica la autenticidad del certificado.
5. **Cifrado SSL/TLS:** Protege la transferencia de datos.

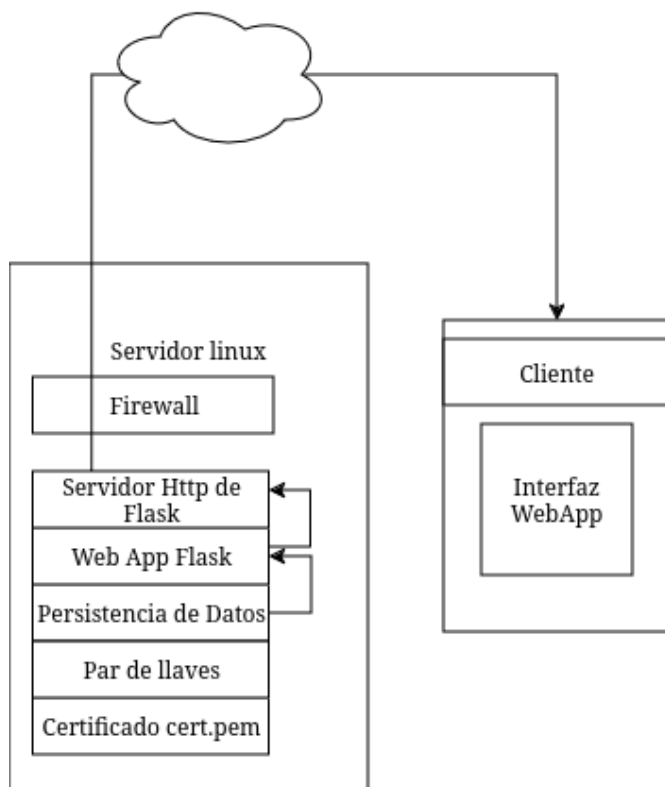


El servidor y el cliente establecen lo que se conoce como “TLS Handshake” [9].



Arquitectura necesaria para su implementación

Para la implementación básica de https se propone una arquitectura con certificado auto firmado. Un certificado auto firmado permite la comunicación cifrada de cliente servidor sin la necesidad de ser aprobado por una entidad certificadora [10].



La implementación consta de lo siguiente:

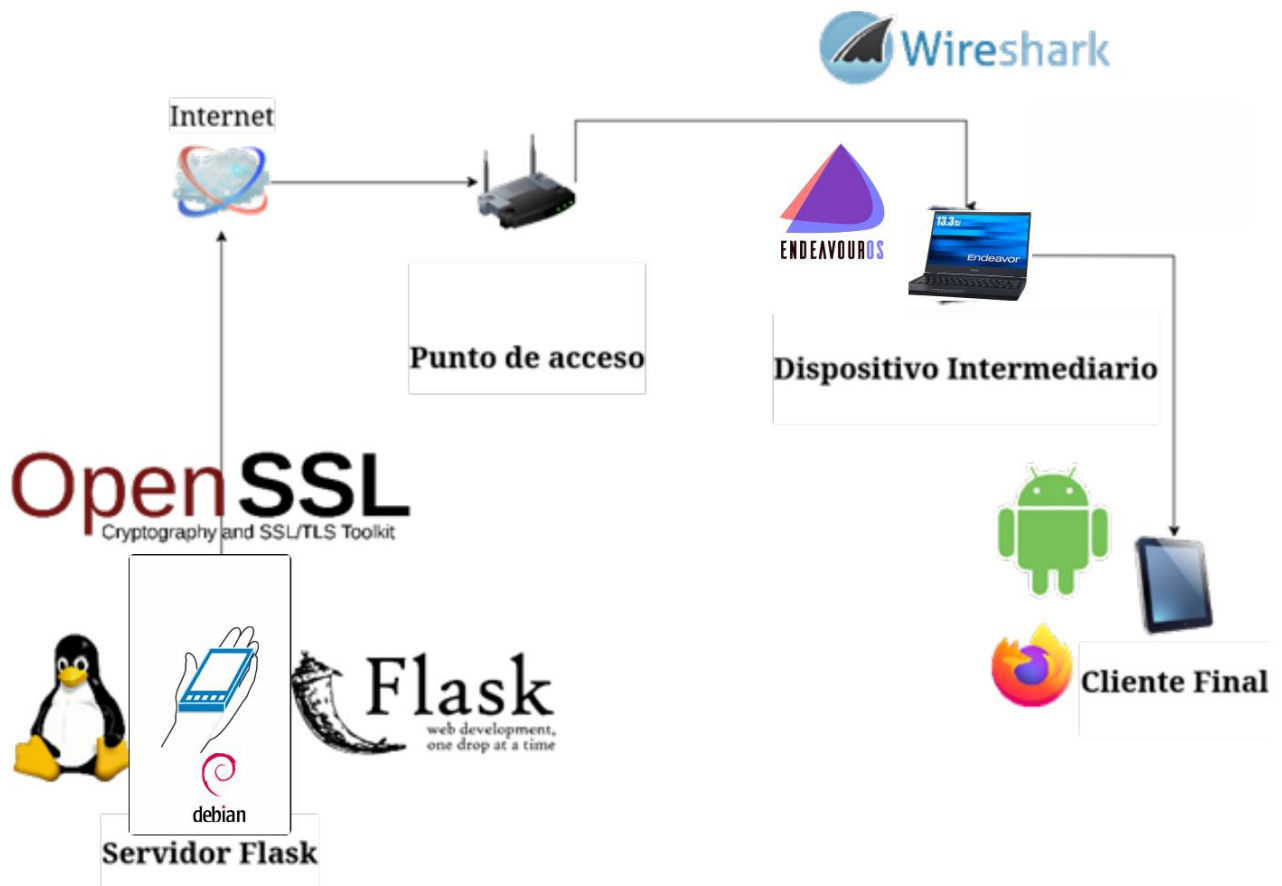
Server Side:

- Web App construida en Flask.
- Servidor http de desarrollo de Flask.
- Persistencia de datos local simple.
- Par de llaves generado con openssl.
- Certificado Generado con openssl.

Client Side:

- Interfaz Gráfica WebApp

Para la demostración de http/https frente un ataque "man in the middle" se propone la siguiente arquitectura.



La implementación se encuentra en el repositorio:

[GRUPO05/TAREA05-U02-G05 at main · 24-25-CSI/GRUPO05](https://github.com/GRUPO05/TAREA05-U02-G05)

Bibliografía

- [1] Cloudflare, "¿Qué es HTTPS?", Cloudflare, 2024. [En línea]. Disponible en: <https://www.cloudflare.com/es-es/learning/ssl/what-is-https/>. [Accedido: 5-dic-2024].
- [2] Cloudflare, "What is SSL?", Cloudflare, 2024. [Online]. Available: <https://www.cloudflare.com/learning/ssl/what-is-ssl/>. [Accessed: 05-Dec-2024].
- [3] Cloudflare, "¿Qué es Transport Layer Security?", Cloudflare, 2024. [En línea]. Disponible en: <https://www.cloudflare.com/es-es/learning/ssl/transport-layer-security-tls/>. [Accedido: 5-dic-2024].
- [4] EDteam, "¿Qué es y cómo funciona HTTPS?", YouTube, 1 de diciembre de 2023. [En línea]. Disponible en: <https://www.youtube.com/watch?v=60606AHuq8c>. [Accedido: 5-dic-2024].
- [5] A. Labinsky, "Features of cryptographic protocols," 2024. doi: 10.61260/2307-7476-2024-1-53-59.
- [6] E. Barker, A. Roginsky, y R. Davis, "Recommendation for Cryptographic Key Generation," NIST Special Publication 800-133, Rev. 2, 2020. [En línea]. Disponible en: <https://doi.org/10.6028/NIST.SP.800-133r2>. [Accedido: 5-dic-2024].
- [7] L. Leonardi, G. Lettieri, P. Perazzo, y S. Saponara, "On the Hardware–Software Integration in Cryptographic Accelerators for Industrial IoT," Applied Sciences, vol. 12, no. 19, p. 9948, 2022. [En línea]. Disponible en: <https://www.mdpi.com/2076-3417/12/19/9948>. [Accedido: 5-dic-2024].
- [8] Q. Scheitle, T. Chung, J. Amann, O. Gasser, L. Brent, G. Carle, R. Holz, J. Hiller, J. Naab, R. van Rijswijk-Deij, O. Hohlfeld, D. Choffnes, and A. Mislove, "Measuring Adoption of Security Additions to the HTTPS Ecosystem," 2018. doi: 10.1145/3232755.3232756.
- [9] G. King and H. Wang, "HTTTPA: HTTPS Attestable Protocol," arXiv (Cornell University), Jan. 2021, doi: <https://doi.org/10.48550/arxiv.2110.07954>.
- [10] M. Grinberg, "Running Your Flask Application Over HTTPS," blog.miguelgrinberg.com, Jun. 04, 2017. <https://blog.miguelgrinberg.com/post/running-your-flask-application-over-https>